DeepStream → VLM (MiniCPM-o) Flow — Trigger & Narration

Detailed step-by-step: frame capture inside DeepStream, through to model analysis, back to alert/evidence output. Covers both live paths.

Path	Status	What it does
A. Trigger analysis	Live today (Qwen2.5-VL) → MiniCPM-o swap planned, config-only	Fires once per tracked object when FSM hits ENTER/VIOLATION. One-shot crop analysis, synchronous request/response.
B. Continuous narration	Planned, not yet live	Always-on scene narration per camera, independent of FSM state. Free-text + structured anomaly flags, feeds a human review queue.

Path A — Trigger Analysis (DeepStream → VLM → Alert)

Step 1 — Detection & tracking (inside `pipeline` container)

RTSP frame batch → `nvinfer` (RF-DETR TensorRT) produces bbox+class+confidence per object → `nvtracker` (ByteTrack) assigns a persistent `track_id` → `nvdsanalytics` checks the box centroid against the camera's ROI polygon, drops anything outside the zone.

Step 2 — FSM state update

`event_fsm.py: EventFSMManager.update()` runs per track, per frame. Track entering ROI → `ENTER`. If it dwells past the configured threshold (default 5s) → `OBSERVE`. Past 15s dwell, or a suspicious-motion signal → `VIOLATION`.
This state machine is the only thing that decides *whether* Path A fires — no VLM call happens without a state transition.

Step 3 — Frame capture (parallel branch, already running)

Independently of the FSM, DeepStream's tee Branch 3 runs `capture_frame_probe()` on every buffer: maps the RGBA buffer, resizes to max 1280px wide, JPEG-encodes at quality 70, caches it per `source_id` in `ctx["frame_jpeg_cache"]`. This cache is what both Path A and Path B read from — no re-decoding.

Step 4 — Trigger fires

In `extract_metadata()` (Branch 2 probe), on FSM transition to `ENTER` (or `VIOLATION` if the ENTER-fire was skipped), the pipeline calls:

```
vlm_grpc_client.py: VLMGrpcClient.submit(
    camera_id, track_id, class_name, dwell_time_s,
    path_summary, trigger_bbox, jpeg_bytes
)
```

This is **fire-and-forget** — handed to a 4-worker thread pool (`_call`) so the GStreamer streaming thread never blocks. A 30s circuit breaker opens after 5 consecutive gRPC failures.

Step 5 — gRPC Analyze call

`AnalyzeRequest` proto sent to `vlm-worker` container on port `50061`:

```
AnalyzeRequest {
  camera_id, track_id, track_session_id, zone_name,
  dwell_time_s, class_name, path_summary,
  trigger_bbox { x, y, w, h },
  frames_jpeg: [bytes],        // the cropped/full frame(s)
  frame_timestamps: [float],
  request_id
}
```

Step 6 — Server-side crop + request build

`grpc_server.py: VLMServicer.Analyze()` receives the request. If only one full frame + bbox arrived (no pre-crop), it calls `VLMWorker._crop_to_bbox()` to derive a cropped close-up, and sends **both** the crop and the full frame to the model (2 images: "suspect object" + "scene context"). If no frame at all was supplied, it falls back to grabbing a live RTSP frame via a persistent ffmpeg grabber.

Step 7 — Prompt build + model call

`client.py: VLMClient.analyze()` builds a system prompt (`build_prompt()` : camera/zone/track/dwell/class/path context) and sends an OpenAI-style chat completion:

```
POST {vlm_base_url}/chat/completions
{
  "model": "openbmb/MiniCPM-o-4_5",
  "max_tokens": 512, "temperature": 0.1,
  "chat_template_kwargs": { "enable_thinking": false },
  "messages": [
    { "role": "system", "content": "" },
    { "role": "user", "content": [
        { "type": "image_url", "image_url": {"url": ""} },
        { "type": "image_url", "image_url": {"url": ""} },
        { "type": "text", "text": "Image 1 is a CROPPED close-up... Image 2 is the full scene..." }
    ]}
  ]
}
```

`enable_thinking=false` is set specifically because MiniCPM-o has a reasoning/CoT mode — this trigger path is synchronous and latency-sensitive, so thinking mode is disabled to keep response time comparable to Qwen2.5-VL.

Step 8 — Response parse (with retry)

`parse_vlm_response()` expects strict JSON: `person_description, action, is_dumping, waste_type, severity, confidence, reasoning, command, deterrent_message` . If parsing fails, one retry is made with a stricter suffix prompt ("Respond with ONLY valid JSON"). If that also fails, a fallback `VLMResult` is returned (confidence=0, severity=warning, command="monitor") — the pipeline never crashes on a bad model response.

Step 9 — Publish result

Server publishes to Kafka `wastewatch.vlm.results` , with `track_context` (path_summary, dwell_time_s, class_name) injected so downstream alert gating doesn't need to round-trip to the pipeline. gRPC also returns an `AnalyzeResponse` synchronously to the caller (unused by the fire-and-forget pipeline client, but used if something calls Analyze directly).

Step 10 — Alert fan-out

`alert-svc: AlertDispatcher._handle_message()` consumes `vlm.results` . Skips if `severity=info` and `is_dumping=false` . Otherwise: throttle check → Kafka `wastewatch.alerts` → fan-out to Telegram, WebSocket (live dashboard), SIP call (if escalated), IP speaker (TTS of `deterrent_message`).

Path B — Continuous Narration planned

This path does not depend on the FSM at all — it runs continuously per active camera, independent of whether any track is currently ENTER/OBSERVE/VIOLATION. Its job is to catch what the box-based detector structurally can't: things without a trackable bbox (a dumping motion, an uncovered load passing through, anything unclassified).

Step 1 — Sampling tap

Reuses the **same** `frame_jpeg_cache` Path A's Step 3 populates — no new GStreamer tee/queue element, no re-running detection. A GLib timer fires `narration_tick()` at an interval derived from `WW_NARRATION_FPS` (e.g. 1fps), reading whatever's currently cached per `source_id` .

Step 2 — Per-camera session submit

```
narration_sampler.py: narration_tick(ctx)
  for source_id, jpeg_bytes in frame_jpeg_cache:
      client = narration_clients[source_id]   # one NarrationClient per camera
      result = client.submit_frame(jpeg_bytes=jpeg_bytes)
```

Each camera has its own long-lived `NarrationClient` — this matters because the real backend (once swapped in) is a **stateful duplex session**, not a stateless request. Mixing frames from different cameras into one session would corrupt the model's rolling context.

What happens inside the model (real MiniCPM-o backend, once swapped in)

```
Omni-Flow duplex streaming (vllm-omni):

  video frame(s) —→ aligned to a shared 1-SECOND CHUNK timeline
                    (paper found 0.1-0.2s chunks starve context — 1s is the floor)
                    "TAIL" (Time-Aligned Interleaving) keeps the output in sync
                    with the input timeline instead of drifting

  each 1s chunk of video → compressed to 64 vision tokens (16x compression)
  [audio path unused here — init_audio=False, vision-only]

  session.streaming_prefill(chunk)   → feeds this chunk into the running context
  session.streaming_generate()       → pulls narration text + structured flags
                                       for that chunk

  This is the SAME session object across calls — context accumulates.
  It is NOT a fresh /v1/chat/completions request each time (that's Path A's
  shape, not this one).
```

Today's plan implements `NarrationClient._narrate()` as a stub returning `{"narration": "", "flags": []}` — the block above is what it becomes once the real vllm-omni endpoint is validated and swapped in. Callers (`narration_tick`) don't change either way — the `submit_frame() → {"narration", "flags"}` contract is fixed.

Step 3 — Flag detection

Model is prompted to free-narrate continuously and emit structured flags alongside the narration text:

```
{
  "narration": "Person tosses a bag over the fence near the bin area.",
  "flags": [
    { "type": "dumping_motion", "confidence": 0.77, "description": "tossing motion over fence" }
  ]
}
```

`type` is one of `dumping_motion`, `uncovered_load`, or `anomaly` (open-ended catch-all). No flags → nothing published, loop continues to the next tick.

Step 4 — Publish + persist

Any flag → `kafka_producer.py: send_narration_flag()` → Kafka `wastewatch.narration.flags` → `backend/app/evidence/narration_worker.py: NarrationFlagWorker._process_flag()` resolves camera name→UUID, inserts a row into `narration_flags` with `review_status="pending"`. This is a **separate, lower-trust table** from `violation_events` — free-text narration is noisier than FSM+VLM confirmation, so it needs a human in the loop before becoming an official violation.

Step 5 — Human review → promotion

Admin hits `GET /api/narration-flags?review_status=pending` to see the queue. `POST /narration-flags/{id}/confirm` creates a real `ViolationEvent` row (tagged `vlm_response.source="narration"`) — from that point it flows through the exact same evidence/alert pipeline as an FSM-triggered violation. `POST /narration-flags/{id}/dismiss` closes it with no event created.

Side-by-side: what's actually different between the two paths

Aspect	Path A — Trigger	Path B — Narration
Fires on	FSM state transition (ENTER/VIOLATION)	Timer tick, every active camera, all the time
Model call shape	Stateless `/v1/chat/completions`, one-shot	Stateful duplex session, `streaming_prefill` / `generate`
Context	None carried between calls — each is independent	Rolling 1s-chunk context per camera session
Thinking mode	Disabled (`enable_thinking=false`) — latency sensitive	N/A — narration mode, not chat-completion reasoning
Output destination	`wastewatch.vlm.results` → alert dispatcher → immediate alert	`wastewatch.narration.flags` → review queue → alert only after human confirms

DB table	`violation_events` (trusted, direct)	`narration_flags` (pending review) → promotes to `violation_events` on confirm
GPU load pattern	Bursty — one call per FSM transition	Continuous — one session per camera, running always

Both paths share one `vllm-omni` process and one GPU — continuous batching lets bursty trigger calls and long-lived narration sessions interleave, but an unbounded number of narration sessions can still queue-delay a latency-sensitive trigger call. `WW_NARRATION_MAX_CONCURRENT_CAMS` is the safety cap on total narration sessions (deferred to load-testing, not yet tuned).